Below is a **Java + Spring Boot "Principle → Anti-pattern → Fix" mapping with working code examples**.

I'm giving you **10 high-impact mappings** (most common in real microservices). If you want, I can extend this into a **full 30+ mapping pack**.

✅ Java + Spring Boot Mapping Examples (Code)

---

1) DIP (Dependency Inversion)

❌ Anti-pattern: Controller depends directly on implementation

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderServiceImpl orderService = new
OrderServiceImpl(); // ❌hard-coded

    @PostMapping
    public String create(@RequestBody String payload) {
        return orderService.createOrder(payload);
    }
}

class OrderServiceImpl {
    public String createOrder(String payload) {
        return "created";
    }
}
```

✅ Fix: Depend on interface + Constructor DI

```
public interface OrderService {
    String createOrder(String payload);
}

@Service
class OrderServiceImpl implements OrderService {
    @Override
    public String createOrder(String payload) {
        return "created";
    }
}

@RestController
```

```
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderService orderService; // ✅ abstraction

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public String create(@RequestBody String payload) {
        return orderService.createOrder(payload);
    }
}
```

---

2) SRP (Single Responsibility Principle)

❌ Anti-pattern: Service handles validation + persistence + external API

```
@Service
public class PaymentService {

    public String pay(String userId, double amount) {
        if (userId == null || amount <= 0) {
            throw new IllegalArgumentException("Invalid input");
        }
        // store payment in DB (pseudo)
        // call external gateway (pseudo)
        return "PAID";
    }
}
```

✅ Fix: Split responsibilities

```
@Component
class PaymentValidator {
    public void validate(String userId, double amount) {
        if (userId == null || userId.isBlank()) throw new
IllegalArgumentException("userId required");
        if (amount <= 0) throw new IllegalArgumentException("amount
must be > 0");
    }
}

@Component
class PaymentGatewayClient {
    public String charge(String userId, double amount) {
        return "TXN-" + System.currentTimeMillis();
    }
}
```

```
@Repository
class PaymentRepository {
    public void save(String userId, double amount, String txnId) {
        // persist data
    }
}

@Service
class PaymentService {
    private final PaymentValidator validator;
    private final PaymentGatewayClient gateway;
    private final PaymentRepository repo;

    public PaymentService(PaymentValidator validator,
PaymentGatewayClient gateway, PaymentRepository repo) {
        this.validator = validator;
        this.gateway = gateway;
        this.repo = repo;
    }

    public String pay(String userId, double amount) {
        validator.validate(userId, amount);
        String txnId = gateway.charge(userId, amount);
        repo.save(userId, amount, txnId);
        return txnId;
    }
}
```

---

3) Fail Fast + Defensive Programming (Validation Layer)

✅ Fix with Bean Validation

DTO

```
import jakarta.validation.constraints.*;

public class CreateUserRequest {

    @NotBlank(message = "name is required")
    private String name;

    @Email(message = "email must be valid")
    @NotBlank
    private String email;

    @Min(value = 18, message = "age must be >= 18")
    private int age;

    // getters & setters
```

```
}
```

Controller

```
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/users")
public class UserController {

    @PostMapping
    public String create(@Valid @RequestBody CreateUserRequest req) {
        return "OK";
    }
}
```

Error Response (Global Handler)

```
import org.springframework.http.*;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.util.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>>
handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(err -> errors.put(err.getField(),
err.getDefaultMessage()));

        Map<String, Object> body = new HashMap<>();
        body.put("message", "Validation failed");
        body.put("errors", errors);
        return ResponseEntity.badRequest().body(body);
    }
}
```

---

4) Idempotency Principle (Avoid duplicate orders/payments)

❌ Anti-pattern: retries create duplicates

A user clicks “Pay” twice → 2 payments.

✅ **Fix: Use `Idempotency-Key` header + DB tracking**

```
@Entity
public class IdempotencyRecord {
    @Id
    private String key;

    private String responseBody;
    private long createdAt;

    // getters/setters
}
public interface IdempotencyRepository extends
JpaRepository<IdempotencyRecord, String> {}
@Service
public class IdempotencyService {

    private final IdempotencyRepository repo;

    public IdempotencyService(IdempotencyRepository repo) {
        this.repo = repo;
    }

    public Optional<String> getExistingResponse(String key) {
        return
repo.findById(key).map(IdempotencyRecord::getResponseBody);
    }

    public void store(String key, String responseBody) {
        IdempotencyRecord r = new IdempotencyRecord();
        r.setKey(key);
        r.setResponseBody(responseBody);
        r.setCreatedAt(System.currentTimeMillis());
        repo.save(r);
    }
}
@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    private final PaymentService paymentService;
    private final IdempotencyService idempotencyService;

    public PaymentController(PaymentService paymentService,
IdempotencyService idempotencyService) {
        this.paymentService = paymentService;
        this.idempotencyService = idempotencyService;
    }

    @PostMapping
    public ResponseEntity<String> pay(
```

```
            @RequestHeader("Idempotency-Key") String key,
            @RequestParam String userId,
            @RequestParam double amount) {

        return idempotencyService.getExistingResponse(key)
                .map(ResponseEntity::ok)
                .orElseGet(() -> {
                    String txnId = paymentService.pay(userId, amount);
                    idempotencyService.store(key, txnId);
                    return ResponseEntity.ok(txnId);
                });
    }
}
```

---

5) Circuit Breaker (Resilience)

❌ Anti-pattern: infinite retries when dependency fails

✅ Fix: Resilience4j Circuit Breaker

pom.xml

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
```

Service

```
import
io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.stereotype.Service;

@Service
public class InventoryClient {

    @CircuitBreaker(name = "inventory", fallbackMethod =
"fallbackStock")
    public int getStock(String productId) {
        // simulate calling external inventory
        throw new RuntimeException("Inventory service down");
    }

    public int fallbackStock(String productId, Throwable ex) {
        return 0; // fallback default stock
    }
}
```

---

6) Timeout Principle (avoid thread starvation)

✅ Fix using WebClient timeouts

```
import io.netty.channel.ChannelOption;
import io.netty.handler.timeout.ReadTimeoutHandler;
import
org.springframework.http.client.reactive.ReactorClientHttpConnector;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.netty.http.client.HttpClient;

import java.time.Duration;

@Component
public class ProductClient {

    private final WebClient webClient;

    public ProductClient() {
        HttpClient httpClient = HttpClient.create()
                .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 2000)
                .responseTimeout(Duration.ofSeconds(2))
                .doOnConnected(conn -> conn.addHandlerLast(new
ReadTimeoutHandler(2)));

        this.webClient = WebClient.builder()
                .clientConnector(new
ReactorClientHttpConnector(httpClient))
                .baseUrl("http://product-service")
                .build();
    }

    public String getProduct(String id) {
        return webClient.get()
                .uri("/api/products/{id}", id)
                .retrieve()
                .bodyToMono(String.class)
                .block(); // for demo
    }
}
```

---

7) Observability: Correlation ID + Logging

✅ Fix: Filter adds `X-Request-Id`

```
import jakarta.servlet.*;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
```

```
import java.io.IOException;
import java.util.UUID;

@Component
public class RequestIdFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        String requestId = request.getHeader("X-Request-Id");
        if (requestId == null || requestId.isBlank()) {
            requestId = UUID.randomUUID().toString();
        }

        MDC.put("requestId", requestId);
        try {
            chain.doFilter(req, res);
        } finally {
            MDC.remove("requestId");
        }
    }
}
```

Now logs can show `requestId` for tracing.

---

8) No Shared DB (Microservices rule)

❌ Anti-pattern: Service A reads/writes Service B tables

✅ Fix: fetch via API/event or local read model (CQRS)

Example: Service B exposes API, Service A calls it:

```
@Service
public class UserProfileClient {
    private final RestTemplate restTemplate = new RestTemplate();

    public String getProfile(String userId) {
        return restTemplate.getForObject(
                "http://user-service/api/users/" + userId +
"/profile",
                String.class
        );
    }
```

```
}
```

---

9) SAGA pattern (distributed transaction)

❌ Anti-pattern: 2PC distributed transaction

✅ Fix: Orchestrated Saga (simplified)

```
@Service
public class OrderSaga {

    private final PaymentService paymentService;
    private final InventoryService inventoryService;

    public OrderSaga(PaymentService paymentService, InventoryService
inventoryService) {
        this.paymentService = paymentService;
        this.inventoryService = inventoryService;
    }

    public String placeOrder(String userId, String productId) {
        String paymentTxn = paymentService.reserve(userId);
        try {
            inventoryService.reserve(productId);
            paymentService.commit(paymentTxn);
            return "ORDER_CONFIRMED";
        } catch (Exception ex) {
            paymentService.cancel(paymentTxn); // compensation
            return "ORDER_CANCELLED";
        }
    }
}
```

---

10) Feature Flags (Deploy ≠ Release)

✅ Fix: flag controls feature rollout

```
@Service
public class FeatureFlagService {
    public boolean isEnabled(String name) {
        // for demo - in real use config server/launchdarkly/unleash
        return "NEW_CHECKOUT".equals(name);
    }
}
@RestController
@RequestMapping("/api/checkout")
public class CheckoutController {

    private final FeatureFlagService flags;
```

```
    public CheckoutController(FeatureFlagService flags) {
        this.flags = flags;
    }

    @GetMapping
    public String checkout() {
        if (flags.isEnabled("NEW_CHECKOUT")) {
            return "NEW CHECKOUT FLOW";
        }
        return "OLD CHECKOUT FLOW";
    }
}
```

✅ **If you want next (recommended)**

I can create a **complete Spring Boot microservices demo project** that includes:

✅ API Gateway + Auth
✅ 3 microservices (Order/Payment/Inventory)
✅ Kafka events + Outbox pattern
✅ Circuit breaker + retries
✅ OpenTelemetry tracing
✅ Docker Compose + Kubernetes manifests

Just tell me: **Maven or Gradle?**